

DOCUMENTO 07 — Especificação da API REST

Projeto: Lindvior

Versão: 1.0

Status: Aprovado

1. Objetivo

Este documento define a especificação da API REST do Lindvior.

Seu objetivo é estabelecer os contratos de comunicação entre clientes e a aplicação, definindo os recursos expostos, endpoints, métodos HTTP, estruturas de requisição e resposta, códigos de status, convenções de nomenclatura e padrões de utilização da API.

Enquanto:

- o Documento 01 define a visão do produto;
- o Documento 02 define os requisitos funcionais;
- o Documento 03 define as regras de negócio;
- o Documento 04 define o modelo de domínio;
- o Documento 05 define o modelo de dados;
- o Documento 06 define a arquitetura da aplicação;

este documento define como essas funcionalidades serão disponibilizadas por meio da API REST.

As definições apresentadas servirão como referência para implementação dos Controllers, DTOs, validações, documentação OpenAPI e testes de integração da aplicação.

2. Escopo

Este documento contempla:

- recursos da API;
- endpoints;
- métodos HTTP;

- estruturas de requisição;
- estruturas de resposta;
- autenticação;
- autorização;
- códigos HTTP;
- tratamento de erros;
- paginação;
- filtros;
- ordenação;
- convenções de nomenclatura;
- versionamento da API.

Não fazem parte deste documento:

- regras de negócio;
- implementação utilizando Spring Boot;
- implementação dos Controllers;
- implementação dos Services;
- consultas SQL;
- estrutura do banco de dados;
- detalhes internos da arquitetura.

Esses assuntos são definidos nos documentos correspondentes.

3. Organização do Documento

Este documento está organizado nos seguintes capítulos.

4. Visão Geral da API

Apresenta os princípios gerais da API REST e sua organização.

5. Convenções Gerais

Define os padrões utilizados por toda a API.

6. Autenticação e Autorização

Define como ocorre o acesso aos recursos protegidos.

7. Recursos da API

Define todos os recursos disponibilizados pela aplicação.

8. Estruturas de Dados

Define os padrões de Request e Response.

9. Tratamento de Erros

Define o formato padronizado das respostas de erro.

10. Paginação, Filtros e Ordenação

Define os mecanismos utilizados para consultas.

11. Versionamento

Define a estratégia de evolução da API.

12. Boas Práticas

Define os princípios que deverão ser respeitados durante toda a implementação da API.

13. Considerações Gerais

Apresenta os princípios finais da especificação e sua integração com os demais documentos do Lindvior.

4. Visão Geral da API

4.1 Objetivo

Este capítulo apresenta a visão geral da API REST do Lindvior.

Seu objetivo é definir os princípios adotados para comunicação entre clientes e a aplicação, estabelecendo uma interface consistente, previsível e alinhada à arquitetura definida no Documento 06.

A API deverá expor os recursos da aplicação de forma padronizada, permitindo que diferentes clientes consumam suas funcionalidades sem conhecimento da implementação interna do sistema.

4.2 Arquitetura da API

O Lindvior disponibilizará uma API baseada no estilo arquitetural REST (Representational State Transfer).

Os recursos serão expostos por meio de endpoints HTTP organizados por contexto funcional, utilizando verbos HTTP padronizados para representar operações sobre os recursos.

A API será stateless, ou seja, cada requisição deverá conter todas as informações necessárias para seu processamento.

A autenticação será realizada por meio de JSON Web Token (JWT).

4.3 Características da API

A API deverá possuir as seguintes características:

- arquitetura REST;
 - comunicação utilizando HTTP;
 - troca de dados em JSON;
 - autenticação baseada em JWT;
 - versionamento da API;
 - respostas padronizadas;
 - tratamento centralizado de erros;
 - utilização de códigos HTTP apropriados;
 - operações stateless;
 - documentação compatível com OpenAPI.
-

4.4 Organização dos Recursos

Os endpoints serão organizados por recurso.

Cada recurso representará um contexto funcional da aplicação.

Inicialmente, a API será composta pelos seguintes recursos:

- Authentication;
- Users;
- Parking;
- Parking Sectors;
- Parking Spots;
- Parking Sessions;
- Dashboard;
- Reports.

Cada recurso possuirá seus próprios endpoints, estruturas de requisição e respostas padronizadas.

4.5 Estrutura Geral dos Endpoints

Todos os endpoints deverão seguir uma estrutura uniforme.

/api/v1/{resource}

Exemplos:

/api/v1/auth

/api/v1/users

/api/v1/parking

/api/v1/parking-sectors

/api/v1/parking-spots

/api/v1/parking-sessions

/api/v1/dashboard

/api/v1/reports

4.6 Princípios da API

A API deverá seguir os seguintes princípios:

- consistência entre endpoints;
 - nomenclatura uniforme;
 - utilização correta dos métodos HTTP;
 - respostas previsíveis;
 - separação entre recursos;
 - independência da implementação interna;
 - compatibilidade com clientes REST.
-

4.7 Integração com a Arquitetura

A API representa exclusivamente a camada de Interface definida no Documento 06.

Cada endpoint deverá encaminhar sua operação para a Camada de Aplicação, respeitando integralmente a arquitetura da aplicação.

Nenhum endpoint deverá implementar regras de negócio, acessar diretamente o banco de dados ou realizar decisões pertencentes ao domínio.

4.8 Fluxo Geral de uma Requisição

Toda requisição deverá seguir o seguinte fluxo.

Cliente

|

HTTP Request

|

Controller

|

Application Service

|

Domínio

|

Repository

|

Banco de Dados

|

Response DTO

|

HTTP Response

|

Cliente

Esse fluxo deverá ser respeitado por todos os endpoints da aplicação.

4.9 Papel da API

A API constitui a interface oficial de comunicação entre clientes e o Lindvior.

Sua responsabilidade consiste exclusivamente em disponibilizar os recursos definidos pelos requisitos funcionais, respeitando a arquitetura da aplicação e preservando o isolamento das regras de negócio.

Toda evolução da API deverá manter compatibilidade com os documentos oficiais do projeto.

5. Convenções Gerais

5.1 Objetivo

Este capítulo define os padrões utilizados por toda a API REST do Lindvior.

Seu objetivo é garantir consistência entre os recursos expostos, facilitando o consumo da API, sua manutenção e evolução.

Todas as convenções descritas neste capítulo deverão ser respeitadas por todos os endpoints da aplicação.

5.2 Convenções de URL

Os recursos deverão utilizar substantivos no plural.

As URLs não deverão representar ações, mas sim recursos.

Exemplos:

GET /api/v1/users

POST /api/v1/users

GET /api/v1/parking-spots

PUT /api/v1/parking-spots/{id}

DELETE /api/v1/parking-spots/{id}

Não deverão existir URLs como:

/createUser

/deleteParkingSpot

/updateSector

5.3 Convenções de Nomenclatura

Toda a API deverá seguir os seguintes padrões:

- URLs em letras minúsculas;
- palavras separadas por hífen (-);
- identificadores utilizando UUID;
- recursos representados por substantivos;
- nomes consistentes entre Request, Response e recursos.

Exemplos:

/parking-spots

/parking-sectors

/parking-sessions

5.4 Métodos HTTP

Cada operação deverá utilizar o método HTTP correspondente à sua finalidade.

Método	Finalidade
GET	Consultar recursos

POST	Criar recursos
PUT	Atualizar integralmente um recurso
DELETE	Remover logicamente um recurso

5.5 Content-Type

Todas as requisições e respostas deverão utilizar:

`application/json`

Exceções poderão existir apenas para futuras funcionalidades de exportação de arquivos.

5.6 Identificadores

Todos os recursos persistentes deverão utilizar UUID como identificador.

Exemplo:

`/api/v1/users/{id}`

`/api/v1/parking-spots/{id}`

5.7 Versionamento

Todos os endpoints deverão possuir o prefixo da versão.

`/api/v1/`

A evolução da API deverá ocorrer por meio da criação de novas versões quando houver alterações incompatíveis.

5.8 Idempotência

Os métodos HTTP deverão respeitar suas características.

Método	Idempotente
GET	Sim
PUT	Sim
DELETE	Sim
POST	Não

5.9 Estrutura das Respostas

Respostas bem-sucedidas deverão retornar exclusivamente os dados necessários para o cliente.

A API não deverá incluir informações internas da aplicação, detalhes de persistência ou dados irrelevantes para o consumidor do recurso.

5.10 Estrutura das Requisições

As requisições deverão conter apenas os atributos necessários para execução da operação.

Campos calculados, identificadores gerados pela aplicação e informações de infraestrutura não deverão ser enviados pelo cliente.

5.11 Consistência entre Recursos

Todos os recursos deverão seguir os mesmos padrões de organização.

Por exemplo:

GET /resource

GET /resource/{id}

POST /resource

PUT /resource/{id}

DELETE /resource/{id}

Sempre que aplicável, novos recursos deverão seguir exatamente essa organização.

5.12 Princípios das Convenções

API-001

Todos os endpoints deverão seguir a mesma estrutura de URL.

API-002

Os métodos HTTP deverão representar corretamente a operação realizada.

API-003

Os recursos deverão utilizar nomenclatura consistente.

API-004

A API deverá utilizar JSON como formato padrão de comunicação.

API-005

Os identificadores dos recursos deverão utilizar UUID.

API-006

A API deverá permanecer consistente durante toda sua evolução.

API-007

Mudanças incompatíveis deverão resultar em uma nova versão da API.

API-008

As convenções definidas neste capítulo deverão ser aplicadas uniformemente a todos os recursos expostos pela aplicação.

6. Autenticação e Autorização

6.1 Objetivo

Este capítulo define como ocorre o acesso aos recursos protegidos da API REST do Lindvior.

Seu objetivo é estabelecer o processo de autenticação, autorização e proteção dos endpoints da aplicação, garantindo que apenas usuários autenticados e autorizados possam executar operações conforme suas permissões.

6.2 Autenticação

A autenticação da API será baseada em JSON Web Token (JWT).

Após informar credenciais válidas, o usuário receberá um token de acesso que deverá ser enviado em todas as requisições destinadas a recursos protegidos.

A API não manterá sessão entre requisições.

6.3 Login

O processo de autenticação ocorrerá por meio do endpoint:

POST /api/v1/auth/login

A requisição deverá conter:

- e-mail;

- senha.

Após autenticação bem-sucedida, a API retornará um token JWT.

6.4 Envio do Token

O token deverá ser enviado utilizando o cabeçalho HTTP Authorization.

Authorization: Bearer <token>

Todos os endpoints protegidos deverão exigir esse cabeçalho.

6.5 Recursos Públicos

Os seguintes recursos poderão ser acessados sem autenticação:

- login;
- criação do primeiro usuário administrador durante a inicialização da aplicação, quando ainda não existir nenhum usuário cadastrado.

Os demais recursos deverão exigir autenticação.

6.6 Recursos Protegidos

Todos os endpoints administrativos e operacionais deverão exigir autenticação.

Exemplos:

/users

/parking

/parking-sectors

/parking-spots

/parking-sessions

/dashboard

6.7 Autorização

Após a autenticação, a aplicação deverá verificar se o usuário possui permissão para executar a operação solicitada.

As permissões deverão ser definidas de acordo com o perfil do usuário.

A autorização deverá ocorrer antes da execução do caso de uso.

6.8 Perfis de Usuário

Na versão 1.0, a aplicação utilizará os seguintes perfis:

Perfil	Responsabilidade
ADMIN	Administração completa do sistema
OPERATOR	Operação diária da aplicação

Novos perfis poderão ser adicionados em versões futuras.

6.9 Fluxo de Autenticação

O processo de autenticação seguirá o seguinte fluxo.

Cliente

|

Login

|

Authentication Service

|

Validação das Credenciais

|

Geração do JWT

|

Resposta

|

Cliente

6.10 Fluxo de Autorização

Após receber uma requisição autenticada, a aplicação seguirá o fluxo abaixo.

Requisição

|

JWT Filter

|

Validação do Token

|

Identificação do Usuário

|

Verificação das Permissões

|

Controller

Caso qualquer etapa falhe, a requisição deverá ser encerrada antes de alcançar a camada de aplicação.

6.11 Respostas de Segurança

A API deverá utilizar códigos HTTP compatíveis com o resultado da autenticação ou autorização.

Situação	Código HTTP
Credenciais inválidas	401 Unauthorized
Token inválido	401 Unauthorized
Token expirado	401 Unauthorized
Usuário sem permissão	403 Forbidden

6.12 Princípios da Segurança

AUTH-001

Toda autenticação deverá utilizar JWT.

AUTH-002

A API deverá permanecer stateless.

AUTH-003

Todos os recursos protegidos deverão exigir autenticação.

AUTH-004

A autorização deverá ocorrer antes da execução do caso de uso.

AUTH-005

Os perfis de usuário deverão determinar as permissões disponíveis.

AUTH-006

Tokens inválidos ou expirados deverão impedir imediatamente o processamento da requisição.

AUTH-007

As regras de autenticação e autorização deverão ser aplicadas de forma uniforme em toda a API.

7. Recursos da API

7.1 Objetivo

Este capítulo define os recursos disponibilizados pela API REST do Lindvior.

Cada recurso representa um conjunto de funcionalidades pertencentes ao mesmo contexto de negócio, conforme definido nos Documentos 02, 03, 04 e 06.

Os endpoints descritos neste capítulo representam os contratos oficiais da versão 1.0 da API.

7.2 Recurso Authentication

Objetivo

Responsável pela autenticação dos usuários da aplicação.

Endpoints

Método	Endpoint	Descrição
POST	/api/v1/auth/login	Realiza autenticação do usuário

7.3 Recurso Users

Objetivo

Responsável pelo gerenciamento dos usuários do sistema.

Endpoints

Método	Endpoint	Descrição
POST	/api/v1/users	Criar usuário
GET	/api/v1/users	Listar usuários
GET	/api/v1/users/{id}	Consultar usuário
PUT	/api/v1/users/{id}	Atualizar integralmente o usuário
DELETE	/api/v1/users/{id}	Desativar usuário

7.4 Recurso Parking

Objetivo

Responsável pelas informações gerais do estacionamento.

Endpoints

Método	Endpoint	Descrição
GET	/api/v1/parking	Consultar estacionamento
PUT	/api/v1/parking	Atualizar integralmente o estacionamento

Como existe apenas um estacionamento na aplicação, não haverá necessidade de identificador na URL.

7.5 Recurso Parking Sectors

Objetivo

Responsável pelo gerenciamento dos setores do estacionamento.

Endpoints

Método	Endpoint	Descrição
POST	/api/v1/parking-sectors	Criar setor
GET	/api/v1/parking-sectors	Listar setores
GET	/api/v1/parking-sectors/{id}	Consultar setor
PUT	/api/v1/parking-sectors/{id}	Atualizar integralmente o setor
DELETE	/api/v1/parking-sectors/{id}	Desativar setor

7.6 Recurso Parking Spots

Objetivo

Responsável pelo gerenciamento das vagas do estacionamento.

Endpoints

Método	Endpoint	Descrição
POST	/api/v1/parking-spots	Criar vaga
GET	/api/v1/parking-spots	Listar vagas
GET	/api/v1/parking-spots/{id}	Consultar vaga
PUT	/api/v1/parking-spots/{id}	Atualizar vaga
DELETE	/api/v1/parking-spots/{id}	Desativar vaga

7.7 Recurso Parking Sessions

Objetivo

Responsável pela consulta das sessões de estacionamento.

As sessões são criadas e encerradas pelo Motor de Simulação, não havendo endpoints para criação ou encerramento manual.

Endpoints

Método	Endpoint	Descrição
GET	/api/v1/parking-sessions	Listar sessões
GET	/api/v1/parking-sessions/{id}	Consultar sessão

7.8 Recurso Dashboard

Objetivo

Responsável pelo fornecimento das informações utilizadas pelo painel principal da aplicação.

Endpoints

Método	Endpoint	Descrição
GET	/api/v1/dashboard	Dashboard principal
GET	/api/v1/dashboard/sectors	Ocupação por setor
GET	/api/v1/dashboard/statistics	Estatísticas gerais

7.9 Recurso Reports

Objetivo

Responsável pela geração e consulta de relatórios.

Endpoints

Método	Endpoint	Descrição
GET	/api/v1/reports	Listar relatórios disponíveis
GET	/api/v1/reports/occupancy	Relatório de ocupação
GET	/api/v1/reports/sessions	Relatório de sessões
GET	/api/v1/reports/vehicles	Relatório de veículos

7.10 Organização dos Recursos

Todos os recursos deverão seguir a mesma estrutura arquitetural.

Resource

|

Controller

|

Request DTO

|

Service

|

Domínio

|

Repository

|

Response DTO

Essa organização deverá ser uniforme para todos os recursos da aplicação.

7.11 Princípios dos Recursos

RES-001

Cada recurso deverá representar um único contexto funcional.

RES-002

Todos os endpoints deverão utilizar os métodos HTTP apropriados.

RES-003

Os recursos deverão possuir nomenclatura consistente.

RES-004

As operações deverão respeitar as regras de negócio definidas pelo domínio.

RES-005

A API deverá expor apenas os recursos necessários para a versão atual da aplicação.

RES-006

Novos recursos deverão seguir a organização definida neste documento.

8. Estruturas de Dados

8.1 Objetivo

Este capítulo define os padrões utilizados pelas estruturas de requisição e resposta da API REST do Lindvior.

Seu objetivo é garantir uniformidade na comunicação entre clientes e a aplicação, estabelecendo convenções para DTOs, formatos de dados e organização das mensagens trafegadas pela API.

As estruturas descritas neste capítulo representam contratos da API e deverão permanecer independentes das entidades do domínio e da persistência.

8.2 Request DTOs

As estruturas de requisição deverão representar exclusivamente os dados necessários para execução da operação solicitada.

Request DTOs não deverão conter:

- identificadores gerados pela aplicação;
- atributos calculados;
- informações internas do domínio;
- dados de auditoria;
- dados de persistência.

Cada endpoint deverá possuir seu próprio Request DTO sempre que necessário.

8.3 Response DTOs

As estruturas de resposta deverão conter apenas as informações relevantes para o consumidor da API.

Response DTOs não deverão expor:

- entidades JPA;
- detalhes internos da arquitetura;
- atributos técnicos da persistência;
- informações utilizadas exclusivamente pelo domínio.

As respostas deverão representar apenas o contrato público da API.

8.4 Estrutura Geral de Requests

As requisições deverão utilizar objetos JSON.

Exemplo:

```
{
  "name": "...",
  "address": "...",
  "capacity": 500
}
```

Cada recurso poderá definir atributos específicos conforme sua finalidade.

8.5 Estrutura Geral de Responses

As respostas deverão utilizar objetos JSON.

Exemplo:

```
{
  "id": "...",
  "name": "...",
  "address": "...",
  "capacity": 500,
```



```
"active": true
}
```

A estrutura exata dependerá do recurso solicitado.

8.6 Coleções

Operações de listagem deverão retornar coleções de recursos.

Exemplo:

```
[
  {
    "id": "...",
    "name": "..."
  },
  {
    "id": "...",
    "name": "..."
  }
]
```

Quando houver paginação, a estrutura será definida no Capítulo 10.

8.7 Datas e Horários

Todos os campos de data e hora deverão utilizar o padrão ISO-8601.

Exemplo:

2026-07-13T18:45:20

Os formatos deverão permanecer consistentes em toda a API.

8.8 Valores Enumerados

Campos baseados em enumerações deverão utilizar seus respectivos valores textuais.

Exemplo:

```
{  
  "status": "FREE",  
  "role": "ADMIN"  
}
```

Os valores aceitos serão definidos pelas enumerações do domínio.

8.9 Valores Nulos

A API deverá evitar retornar atributos nulos quando esses não agregarem valor ao consumidor.

Sempre que possível:

- atributos obrigatórios deverão estar presentes;
 - atributos opcionais poderão ser omitidos quando não houver informação disponível.
-

8.10 Independência do Domínio

DTOs representam contratos da API.

Eles não deverão reproduzir necessariamente a estrutura das entidades do domínio.

Mudanças internas na implementação não deverão alterar os contratos públicos da API sem necessidade.

8.11 Princípios das Estruturas de Dados

DTO-001

Toda comunicação deverá utilizar JSON.

DTO-002

DTOs deverão representar exclusivamente contratos da API.

DTO-003

Entidades do domínio não deverão ser expostas diretamente.

DTO-004

As estruturas deverão permanecer consistentes entre todos os recursos.

DTO-005

Datas deverão utilizar o padrão ISO-8601.

DTO-006

Valores enumerados deverão utilizar representação textual.

DTO-007

Mudanças internas do domínio não deverão impactar os contratos públicos da API.

9. Tratamento de Erros

9.1 Objetivo

Este capítulo define como a API REST do Lindvior deverá tratar situações de erro.

Seu objetivo é padronizar as respostas retornadas pela aplicação quando uma requisição não puder ser processada com sucesso, garantindo consistência, previsibilidade e facilidade de integração para os consumidores da API.

Todas as respostas de erro deverão seguir um formato único.

9.2 Princípios Gerais

O tratamento de erros deverá observar os seguintes princípios:

- respostas padronizadas;
- utilização correta dos códigos HTTP;
- mensagens claras;
- identificação precisa da causa do erro;
- independência da implementação interna.

Detalhes técnicos da aplicação não deverão ser expostos ao cliente.

9.3 Estrutura da Resposta de Erro

Todas as respostas de erro deverão utilizar uma estrutura JSON padronizada.

Exemplo:

```
{
  "timestamp": "2026-07-13T20:15:30",
  "status": 400,
  "error": "Validation Error",
  "message": "The request contains invalid data.",
  "path": "/api/v1/users"
}
```

Cada erro poderá conter informações adicionais quando necessário.

9.4 Erros de Validação

Quando uma requisição possuir dados inválidos, a API deverá retornar erro de validação.

Exemplo:

```
{
  "timestamp": "2026-07-13T20:15:30",
  "status": 400,
  "error": "Validation Error",
  "message": "One or more fields are invalid.",
  "fieldErrors": [
    {
      "field": "email",
      "message": "must be a valid email"
    },
    {
      "field": "password",
```

```
    "message": "must not be blank"
  }
]
}
```

9.5 Códigos HTTP

A API deverá utilizar os seguintes códigos HTTP.

Código	Situação
200 OK	Operação realizada com sucesso
201 Created	Recurso criado
204 No Content	Operação sem conteúdo de resposta
400 Bad Request	Dados inválidos
401 Unauthorized	Falha de autenticação
403 Forbidden	Usuário sem permissão
404 Not Found	Recurso não encontrado
405 Method Not Allowed	Método HTTP não permitido
409 Conflict	Violação de regra de negócio ou conflito
500 Internal Server Error	Erro inesperado

9.6 Tratamento Centralizado

Todo tratamento de exceções deverá ocorrer de forma centralizada.

A API não deverá implementar tratamento individual em Controllers.

Essa responsabilidade pertencerá ao mecanismo global de tratamento de exceções da aplicação.

9.7 Exceções de Negócio

Quando uma regra de negócio impedir a execução de uma operação, a API deverá retornar uma resposta apropriada.

Exemplos:

- e-mail já cadastrado;
- código da vaga duplicado;
- setor inexistente;
- estacionamento inexistente;
- vaga indisponível.

Essas situações deverão resultar em respostas consistentes e compatíveis com o código HTTP correspondente.

9.8 Exceções Não Tratadas

Erros inesperados deverão gerar resposta padronizada.

Detalhes internos da aplicação, como stack traces, nomes de classes ou consultas SQL, nunca deverão ser retornados ao cliente.

Essas informações deverão permanecer disponíveis apenas nos mecanismos internos de log da aplicação.

9.9 Registro de Erros

Toda exceção deverá ser registrada pelos mecanismos de logging da aplicação.

O nível de detalhamento dos registros deverá ser suficiente para permitir análise, investigação e correção do problema sem expor informações sensíveis aos consumidores da API.

9.10 Princípios do Tratamento de Erros

ERR-001

Toda resposta de erro deverá seguir o formato padronizado definido neste documento.

ERR-002

Os códigos HTTP deverão representar corretamente a natureza do erro.

ERR-003

Erros de validação deverão identificar claramente os campos inválidos.

ERR-004

As regras de negócio deverão produzir respostas consistentes.

ERR-005

Informações internas da aplicação não deverão ser expostas ao cliente.

ERR-006

Toda exceção deverá ser registrada pelos mecanismos de logging da aplicação.

ERR-007

O tratamento de exceções deverá permanecer centralizado durante toda a evolução da API.

10. Paginação, Filtros e Ordenação

10.1 Objetivo

Este capítulo define os mecanismos utilizados pela API REST do Lindvior para consultas de recursos.

Seu objetivo é padronizar a paginação, os filtros e a ordenação das informações retornadas pela API, garantindo consistência entre todos os endpoints de consulta.

Sempre que aplicável, todos os recursos deverão utilizar os padrões definidos neste capítulo.

10.2 Paginação

Recursos que retornarem coleções deverão suportar paginação.

A paginação evita o retorno de grandes volumes de dados em uma única resposta, melhorando o desempenho da aplicação e facilitando o consumo da API.

10.3 Parâmetros de Paginação

A paginação utilizará os seguintes parâmetros.

Parâmetro	Descrição
page	Número da página
size	Quantidade de registros por página
sort	Campo utilizado para ordenação

Exemplo:

GET /api/v1/users?page=0&size=20&sort=email

10.4 Estrutura da Resposta Paginada

As respostas paginadas deverão conter tanto os registros quanto informações sobre a paginação.

Exemplo:


```
{
  "content": [
    {
      "id": "...",
      "name": "..."
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 150,
  "totalPages": 8,
  "first": true,
  "last": false
}
```

10.5 Filtros

Recursos que suportarem consultas deverão permitir filtros compatíveis com seu contexto funcional.

Exemplos:

`/api/v1/users?active=true`

`/api/v1/parking-spots?status=FREE`

`/api/v1/parking-sessions?licensePlate=ABC1234`

`/api/v1/reports?startDate=2026-07-01&endDate=2026-07-31`

Os filtros disponíveis serão definidos individualmente por cada recurso.

10.6 Ordenação

As consultas poderão ser ordenadas por atributos específicos.

Exemplo:

`GET /api/v1/users?sort=email`

`GET /api/v1/users?sort=createdAt`

`GET /api/v1/users?sort=name`

Quando necessário, poderá ser informado o sentido da ordenação.

GET /api/v1/users?sort=name,asc

GET /api/v1/users?sort=createdAt,desc

10.7 Combinação de Parâmetros

Os mecanismos definidos neste capítulo poderão ser utilizados simultaneamente.

Exemplo:

GET /api/v1/parking-spots?page=0&size=30&sort=code&status=FREE

Essa combinação deverá produzir resultados consistentes e previsíveis.

10.8 Recursos Sem Paginação

Nem todos os recursos necessitarão de paginação.

Consultas que retornem pequeno volume de dados poderão devolver listas completas.

Exemplos:

- dashboard;
- informações do estacionamento;
- setores;
- recursos administrativos com quantidade limitada de registros.

A decisão deverá considerar o comportamento esperado de cada recurso.

10.9 Princípios das Consultas

PAG-001

Recursos com grande volume de dados deverão suportar paginação.

PAG-002

Os parâmetros de paginação deverão permanecer consistentes em toda a API.

PAG-003

Filtros deverão respeitar o contexto funcional de cada recurso.

PAG-004

A ordenação deverá utilizar atributos pertencentes ao recurso consultado.

PAG-005

A combinação de paginação, filtros e ordenação deverá produzir resultados previsíveis.

PAG-006

Consultas simples não deverão utilizar paginação quando isso não agregar valor.

11. Versionamento

11.1 Objetivo

Este capítulo define a estratégia de versionamento da API REST do Lindvior.

Seu objetivo é garantir que a evolução da API possa ocorrer de forma controlada, preservando a compatibilidade entre clientes e permitindo a introdução de novas funcionalidades sem comprometer integrações existentes.

11.2 Estratégia de Versionamento

O Lindvior adotará versionamento por URL.

Todas as versões da API deverão possuir um identificador explícito no caminho da requisição.

Exemplo:

/api/v1/users

/api/v1/parking-spots

/api/v1/dashboard

11.3 Versão Inicial

A primeira versão oficial da API será:

v1

Todos os endpoints definidos neste documento pertencem à versão 1 da API.

11.4 Evolução da API

A inclusão de novas funcionalidades compatíveis com a versão atual poderá ocorrer sem alteração da versão da API.

Exemplos:

- inclusão de novos filtros;
- inclusão de novos endpoints;
- novos atributos opcionais nas respostas;
- novas operações compatíveis.

Essas evoluções não deverão quebrar integrações existentes.

11.5 Alterações Incompatíveis

Sempre que uma alteração tornar incompatível o contrato existente, uma nova versão da API deverá ser criada.

Exemplos:

- remoção de endpoints;
- alteração obrigatória da estrutura de Request;
- alteração obrigatória da estrutura de Response;

- mudança no comportamento esperado de um recurso.

Nesses casos, a versão anterior deverá permanecer disponível durante o período de migração.

11.6 Compatibilidade

A evolução da API deverá priorizar compatibilidade retroativa sempre que possível.

Mudanças que afetem consumidores existentes deverão ser evitadas.

Quando inevitáveis, deverão resultar em uma nova versão da API.

11.7 Descontinuação de Versões

Versões antigas poderão ser descontinuadas futuramente.

A remoção de uma versão somente deverá ocorrer após período adequado de transição e comunicação aos consumidores da API.

A versão mais recente passará a representar a implementação oficial da aplicação.

11.8 Princípios do Versionamento

VER-001

Toda versão da API deverá possuir identificador explícito na URL.

VER-002

A primeira versão oficial da API será a versão 1.

VER-003

Novas funcionalidades compatíveis não deverão exigir mudança de versão.

VER-004

Alterações incompatíveis deverão resultar em nova versão da API.

VER-005

A compatibilidade retroativa deverá ser preservada sempre que possível.

VER-006

Versões antigas poderão ser removidas apenas após período adequado de transição.

12. Boas Práticas

12.1 Objetivo

Este capítulo estabelece as boas práticas que deverão ser seguidas durante a implementação da API REST do Lindvior.

Seu objetivo é garantir consistência, legibilidade, previsibilidade e facilidade de manutenção da API, preservando a qualidade dos contratos definidos neste documento.

As recomendações descritas neste capítulo deverão ser aplicadas a todos os recursos da aplicação.

12.2 Consistência

Todos os recursos deverão seguir o mesmo padrão de organização.

Endpoints semelhantes deverão possuir comportamento semelhante.

Essa consistência reduz a curva de aprendizado para consumidores da API e facilita sua evolução.

12.3 Simplicidade

Os contratos da API deverão permanecer simples.

A API deverá expor apenas as informações necessárias para execução de cada operação.

Campos desnecessários, redundantes ou de uso exclusivamente interno não deverão fazer parte dos contratos públicos.

12.4 Clareza

Os nomes utilizados na API deverão ser claros e objetivos.

Recursos, parâmetros, atributos e estruturas deverão representar exatamente o significado das informações transmitidas.

Abreviações desnecessárias deverão ser evitadas.

12.5 Independência da Implementação

A API representa um contrato entre clientes e a aplicação.

Mudanças internas na implementação não deverão impactar esse contrato, salvo quando houver necessidade de evolução da própria API.

Controllers, entidades JPA, banco de dados ou detalhes de infraestrutura não deverão influenciar a estrutura dos recursos expostos.

12.6 Padronização

Todos os recursos deverão utilizar:

- mesma estrutura de URLs;
- mesma organização de endpoints;
- mesmas convenções de nomenclatura;
- mesmas estruturas de erro;
- mesmos padrões de autenticação;
- mesmos critérios de paginação.

Essa padronização deverá ser mantida durante toda a evolução da aplicação.

12.7 Evolução

Novos recursos deverão seguir exatamente os padrões definidos neste documento.

A introdução de novas funcionalidades não deverá comprometer a organização da API existente.

12.8 Documentação

Toda alteração realizada na API deverá ser refletida neste documento.

A documentação deverá permanecer sincronizada com a implementação.

A especificação da API constitui a referência oficial para consumidores e desenvolvedores da aplicação.

12.9 Princípios das Boas Práticas

BP-001

Toda a API deverá permanecer consistente.

BP-002

Os contratos deverão permanecer simples e objetivos.

BP-003

Mudanças internas da aplicação não deverão alterar os contratos públicos sem necessidade.

BP-004

Toda nova funcionalidade deverá seguir os padrões definidos neste documento.

BP-005

A documentação da API deverá permanecer atualizada durante toda a evolução do projeto.

BP-006

A API deverá priorizar legibilidade, previsibilidade e facilidade de integração.

13. Considerações Gerais

13.1 Objetivo

Este capítulo apresenta as considerações finais sobre a especificação da API REST do Lindvior.

Seu objetivo é consolidar os princípios definidos ao longo deste documento e estabelecer a API como o contrato oficial de comunicação entre clientes e a aplicação.

As definições aqui apresentadas deverão orientar toda a implementação dos endpoints da versão 1.0.

13.2 Papel da API

A API REST representa a interface oficial de acesso ao Lindvior.

Sua responsabilidade consiste em disponibilizar as funcionalidades definidas pelos requisitos funcionais por meio de contratos consistentes, padronizados e independentes da implementação interna da aplicação.

Toda comunicação entre clientes e o sistema deverá ocorrer através da API definida neste documento.

13.3 Integração com os Demais Documentos

O Documento 07 complementa os documentos anteriormente definidos.

Sua relação com os demais documentos pode ser resumida da seguinte forma:

- **Documento 01** — define a visão do produto.
- **Documento 02** — define os requisitos funcionais.
- **Documento 03** — estabelece as regras de negócio.
- **Documento 04** — define o modelo de domínio.
- **Documento 05** — define o modelo de dados.
- **Documento 06** — define a arquitetura da aplicação.
- **Documento 07** — define os contratos oficiais da API REST.

Todos os documentos deverão permanecer consistentes entre si.

13.4 Utilização Durante o Desenvolvimento

Durante a implementação do Lindvior, este documento deverá servir como referência para:

- implementação dos Controllers;
- definição dos endpoints;
- criação dos Request DTOs;
- criação dos Response DTOs;
- implementação das validações da API;
- tratamento de erros;
- documentação OpenAPI;
- testes de integração.

Sempre que houver dúvidas sobre o comportamento da API, este documento deverá ser considerado a referência oficial.

13.5 Evolução da API

A API poderá evoluir ao longo do desenvolvimento da aplicação.

Toda evolução deverá:

- preservar a compatibilidade sempre que possível;
- respeitar as convenções definidas neste documento;
- manter consistência entre recursos;
- ser documentada antes de sua implementação.

Alterações incompatíveis deverão resultar em uma nova versão da API.

13.6 Considerações Finais

A especificação apresentada neste documento representa o contrato oficial da API REST do Lindvior.

Seu propósito é garantir uma interface consistente, previsível e independente da implementação interna da aplicação, permitindo que clientes consumam os recursos do sistema de forma padronizada.

Ao definir convenções, estruturas de dados, mecanismos de autenticação, tratamento de erros, paginação, versionamento e organização dos recursos, este documento reduz ambiguidades durante o desenvolvimento e facilita tanto a implementação quanto a manutenção da API.

Toda implementação da API REST do Lindvior deverá utilizar este documento como referência para definição de seus contratos públicos.